

Implement playwright/typescript with vibe coding

The screenshot displays a web application interface for managing contacts. The top navigation bar includes links for Profile, Shortcuts, and Logout. A search bar is present next to a user profile icon. The main content area features a sidebar menu with options like Opportunities, Quotes, Orders, Forecasts, Marketing, Partners, Teams, Reports, and opentaps. Below the menu, there's a header section with tabs for Hari R (TestLeaf), Hari R (TestLeaf), Hari R (TestLeaf), and Vani Dhandapani (Qeagle). The central part of the page contains a form for creating a contact, with fields for Last name, Last Name (Local), Birth Date, Department, Phone Number, Person to Ask For, Attention Name, Address Line 2, State/Province, and Country. A dropdown menu for Currency is set to USD - American Dollar. At the bottom left, there's a 'Create Contact' button. On the right side, a settings panel is visible, showing options for Code Generator, Settings, Supported Language (TypeScript), Browser Engine (Playwright), Select LLM Provider (Groq), Select Model (llama-3.3-70b-versatile), and Groq API Key.

DOM selected:

On LeafTaps account create contact with all possible positive and negative test cases

Playwright + Typescript code:

Feature: Create Contact

As a user

I want to create a contact

So that I can store contact information

Scenario Outline: Create contact with valid details

Given I am on the create contact page

When I enter "<firstName>" as first name

And I enter "<lastName>" as last name

And I enter "<firstNameLocal>" as first name local

And I enter "<lastNameLocal>" as last name local

And I enter "<salutation>" as salutation

And I enter "<birthDate>" as birth date

And I enter "<title>" as title

And I enter "<department>" as department

And I enter "<preferredCurrency>" as preferred currency

And I enter "<initialAccount>" as initial account

And I enter "<description>" as description

And I enter "<importantNote>" as important note

And I enter "<countryCode>" as country code

And I enter "<areaCode>" as area code

And I enter "<phoneNumber>" as phone number

And I enter "<extension>" as extension

And I enter "<askForName>" as ask for name
And I enter "<email>" as email
And I enter "<toName>" as to name
And I enter "<attentionName>" as attention name
And I enter "<address1>" as address line 1
And I enter "<address2>" as address line 2
And I enter "<city>" as city
And I select "<state>" as state
And I enter "<zipCode>" as zip code
And I select "<country>" as country
And I enter "<zipExtension>" as zip extension
Then I should see the contact created successfully

Examples:

| firstName | lastName | firstNameLocal | lastNameLocal | salutation | birthDate | title
| department | preferredCurrency | initialAccount | description | importantNote |
countryCode | areaCode | phoneNumber | extension | askForName | email | toName |
attentionName | address1 | address2 | city | state | zipCode | country | zipExtension |

| Ramesh | Kumar | ரமேஷ் | குமார் | Mr. | 01/01/1990 | Manager | Sales
| USD | 12345 | Test | Test | 1 | 123 | 1234567890 | 123 | Test |
test@example.com | Ramesh Kumar | Test | Test | Test | Chennai | Tamil Nadu
| 600001 | India | 123 |

| Suresh | Kumar | சுரேஷ் | குமார் | Mr. | 01/01/1991 | Manager | Sales
| USD | 12345 | Test | Test | 1 | 123 | 1234567890 | 123 | Test |
test@example.com | Suresh Kumar | Test | Test | Test | Chennai | Tamil Nadu |
600001 | India | 123 |

Scenario Outline: Create contact with invalid details

Given I am on the create contact page
When I enter "<firstName>" as first name
And I enter "<lastName>" as last name

And I enter "<firstNameLocal>" as first name local
And I enter "<lastNameLocal>" as last name local
And I enter "<salutation>" as salutation
And I enter "<birthDate>" as birth date
And I enter "<title>" as title
And I enter "<department>" as department
And I enter "<preferredCurrency>" as preferred currency
And I enter "<initialAccount>" as initial account
And I enter "<description>" as description
And I enter "<importantNote>" as important note
And I enter "<countryCode>" as country code
And I enter "<areaCode>" as area code
And I enter "<phoneNumber>" as phone number
And I enter "<extension>" as extension
And I enter "<askForName>" as ask for name
And I enter "<email>" as email
And I enter "<toName>" as to name
And I enter "<attentionName>" as attention name
And I enter "<address1>" as address line 1
And I enter "<address2>" as address line 2
And I enter "<city>" as city
And I select "<state>" as state
And I enter "<zipCode>" as zip code
And I select "<country>" as country
And I enter "<zipExtension>" as zip extension
Then I should see an error message

Examples:

| firstName | lastName | firstNameLocal | lastNameLocal | salutation | birthDate | title
| department | preferredCurrency | initialAccount | description | importantNote |
countryCode | areaCode | phoneNumber | extension | askForName | email | toName |
attentionName | address1 | address2 | city | state | zipCode | country | zipExtension |

| | Kumar | ரமேஷ் | குமார் | Mr. | 01/01/1990 | Manager | Sales | USD
| 12345 | Test | Test | 1 | 123 | 1234567890 | 123 | Test |
test@example.com | Ramesh Kumar | Test | Test | Test | Chennai | Tamil Nadu
| 600001 | India | 123 |

| Suresh | | சுரேஷ் | குமார் | Mr. | 01/01/1991 | Manager | Sales |
USD | 12345 | Test | Test | 1 | 123 | 1234567890 | 123 | Test |
test@example.com | Suresh Kumar | Test | Test | Test | Chennai | Tamil Nadu |
600001 | India | 123 |

```plaintext

```typescript

// Page Object: createContactPage.ts

import { Page } from '@playwright/test';

export class CreateContactPage {

constructor(private page: Page) {}

async goto() {

await this.page.goto('https://leaftaps.com/crmsfa/control/createContactForm');

}

async enterFirstName(firstName: string) {

await this.page.fill('#firstNameField', firstName);

}

```
async enterLastName(lastName: string) {  
  await this.page.fill('#lastNameField', lastName);  
}
```

```
async enterFirstNameLocal(firstNameLocal: string) {  
  await this.page.fill('#createContactForm_firstNameLocal', firstNameLocal);  
}
```

```
async enterLastNameLocal(lastNameLocal: string) {  
  await this.page.fill('#createContactForm_lastNameLocal', lastNameLocal);  
}
```

```
async enterSalutation(salutation: string) {  
  await this.page.fill('#createContactForm_personalTitle', salutation);  
}
```

```
async enterBirthDate(birthDate: string) {  
  await this.page.fill('#createContactForm_birthDate', birthDate);  
}
```

```
async enterTitle(title: string) {  
  await this.page.fill('#createContactForm_generalProfTitle', title);  
}
```

```
async enterDepartment(department: string) {  
  await this.page.fill('#createContactForm_departmentName', department);  
}
```

```
async enterPreferredCurrency(preferredCurrency: string) {  
    await this.page.selectOption('#createContactForm_preferredCurrencyUomId',  
preferredCurrency);  
}
```

```
async enterInitialAccount(initialAccount: string) {  
    await this.page.fill('#createContactForm_accountPartyId', initialAccount);  
}
```

```
async enterDescription(description: string) {  
    await this.page.fill('#createContactForm_description', description);  
}
```

```
async enterImportantNote(importantNote: string) {  
    await this.page.fill('#createContactForm_importantNote', importantNote);  
}
```

```
async enterCountryCode(countryCode: string) {  
    await this.page.fill('#createContactForm_primaryPhoneCountryCode', countryCode);  
}
```

```
async enterAreaCode(areaCode: string) {  
    await this.page.fill('#createContactForm_primaryPhoneAreaCode', areaCode);  
}
```

```
async enterPhoneNumber(phoneNumber: string) {  
    await this.page.fill('#createContactForm_primaryPhoneNumber', phoneNumber);  
}
```

```
async enterExtension(extension: string) {  
  await this.page.fill('#createContactForm_primaryPhoneExtension', extension);  
}
```

```
async enterAskForName(askForName: string) {  
  await this.page.fill('#createContactForm_primaryPhoneAskForName', askForName);  
}
```

```
async enterEmail(email: string) {  
  await this.page.fill('#createContactForm_primaryEmail', email);  
}
```

```
async enterToName(toName: string) {  
  await this.page.fill('#generalToNameField', toName);  
}
```

```
async enterAttentionName(attentionName: string) {  
  await this.page.fill('#createContactForm_generalAttnName', attentionName);  
}
```

```
async enterAddress1(address1: string) {  
  await this.page.fill('#createContactForm_generalAddress1', address1);  
}
```

```
async enterAddress2(address2: string) {  
  await this.page.fill('#createContactForm_generalAddress2', address2);  
}
```



```
async enterCity(city: string) {  
  await this.page.fill('#createContactForm_generalCity', city);  
}
```

```
async selectState(state: string) {  
  await this.page.selectOption('#createContactForm_generalStateProvinceGeold',  
state);  
}
```

```
async enterZipCode(zipCode: string) {  
  await this.page.fill('#createContactForm_generalPostalCode', zipCode);  
}
```

```
async selectCountry(country: string) {  
  await this.page.selectOption('#createContactForm_generalCountryGeold', country);  
}
```

```
async enterZipExtension(zipExtension: string) {  
  await this.page.fill('#createContactForm_generalPostalCodeExt', zipExtension);  
}
```

```
async submitForm() {  
  await this.page.click('input[type="submit"]');  
}
```

```
async isContactCreated() {  
  return this.page.isVisible('#contactCreated');
```

```
}
```

```
async getErrorMessage() {  
  return this.page.textContent('#error_message');  
}  
}
```

```
```plaintext
```

```
```typescript
```

```
// Test: createContact.spec.ts
```

```
import { test, expect } from '@playwright/test';
```

```
import { CreateContactPage } from './createContactPage';
```

```
test.describe('Create Contact Feature', () => {  
  test('Create contact with valid details', async ({ page }) => {  
    const createContactPage = new CreateContactPage(page);  
    await createContactPage.goto();  
    await createContactPage.enterFirstName('Ramesh');  
    await createContactPage.enterLastName('Kumar');  
    await createContactPage.enterFirstNameLocal('ரமேஷ்');  
    await createContactPage.enterLastNameLocal('குமார்');  
    await createContactPage.enterSalutation('Mr.');
```

```

    await createContactPage.enterInitialAccount('12345');
    await createContactPage.enterDescription('Test');
    await createContactPage.enterImportantNote('Test');
    await createContactPage.enterCountryCode('1');
    await createContactPage.enterAreaCode('123');
    await createContactPage.enterPhoneNumber('1234567890');
    await createContactPage.enterExtension('123');
    await createContactPage.enterAskForName('Test');
    await createContactPage.enterEmail('test@example.com');
    await createContactPage.enterToName('Ramesh Kumar');
    await createContactPage.enterAttentionName('Test');
    await createContactPage.enterAddress1('Test');
    await createContactPage.enterAddress2('Test');
    await createContactPage.enterCity('Chennai');
    await createContactPage.selectState('Tamil Nadu');
    await createContactPage.enterZipCode('600001');
    await createContactPage.selectCountry('India');
    await createContactPage.enterZipExtension('123');
    await createContactPage.submitForm();
    expect(await createContactPage.isContactCreated()).toBeTruthy();
  });

```

```

test('Create contact with invalid details', async ({ page }) => {
  const createContactPage = new CreateContactPage(page);
  await createContactPage.goto();
  await createContactPage.enterFirstName('');
  await createContactPage.enterLastName('Kumar');
  await createContactPage.enterFirstNameLocal('ரமேஷ்');

```

```
await createContactPage.enterLastNameLocal('சுமார்');
await createContactPage.enterSalutation('Mr. ');
await createContactPage.enterBirthDate('01/01/1990');
await createContactPage.enterTitle('Manager');
await createContactPage.enterDepartment('Sales');
await createContactPage.enterPreferredCurrency('USD');
await createContactPage.enterInitialAccount('12345');
await createContactPage.enterDescription('Test');
await createContactPage.enterImportantNote('Test');
await createContactPage.enterCountryCode('1');
await createContactPage.enterAreaCode('123');
await createContactPage.enterPhoneNumber('1234567890');
await createContactPage.enterExtension('123');
await createContactPage.enterAskForName('Test');
await createContactPage.enterEmail('test@example.com');
await createContactPage.enterToName('Ramesh Kumar');
await createContactPage.enterAttentionName('Test');
await createContactPage.enterAddress1('Test');
await createContactPage.enterAddress2('Test');
await createContactPage.enterCity('Chennai');
await createContactPage.selectState('Tamil Nadu');
await createContactPage.enterZipCode('600001');
await createContactPage.selectCountry('India');
await createContactPage.enterZipExtension('123');
await createContactPage.submitForm();

expect(await createContactPage.getErrorMessage()).toContain('First name is
required');

});
```

});